Chapter 1

# Why agreeing is hard

You have built a container runtime, so somewhere in your system there is state that other programs depend on: which containers exist, their IDs, their resource limits. Today that state probably sits in one place. If that machine reboots, every program that consults the state stalls until it comes back, and if it dies, everything depending on it breaks. The obvious move is to store the state on three servers instead of one, so the loss of any single machine costs you a copy, not the service.

Then the next request arrives, `create container "web-1"`, and a humble question turns vicious. The three servers each heard about the request through a network that can lose, delay, or reorder messages. Which server actually decides? What happens when the network makes two servers disagree about whether the deciding server is even alive? This chapter is the story of why "just copy the data" does not end the problem but starts it.

> THE CRUX
> *Why can't a cluster of servers that each saw the same requests just agree on what happened, and what is the least we must assume so that they can agree at all?*

## Copying the data inherits the agreement problem

**Replication** sounds like a backup plan, and it does solve the narrow problem it advertises: if one disk or one machine dies, the data still exists elsewhere. Replication is what you reach for when you want service with no downtime even if a machine fails, and it handles the clean case well, the failure of a single replica that simply stops[1]. The trouble is that your registry is not a pile of bytes. It is a live map that changes as containers are created and deleted, and now every change has to land on every copy.

**replication** Keeping identical copies of some state on several machines so that the loss of one machine does not lose the state.

The moment there is more than one copy and more than one client, copying becomes an agreement problem, and the agreement that is needed is a single order of operations, stronger than agreeing on each value separately. Two clients create and delete the same container name at nearly the same time, and the three servers must end with the same final map, which means they must all apply the two operations in the same order.

The standard way to think about this is the **replicated state machine**. State machines on a collection of servers compute identical copies of the same state and keep operating even when some of the servers are down[2]. Real systems use this shape constantly, often without naming it: large-scale systems with a single cluster **leader**, such as the Google File System (GFS), the Hadoop Distributed File System (HDFS), and RAMCloud, keep a separate replicated state machine to manage leader election and store configuration that must survive leader crashes, and Chubby and ZooKeeper are examples of that shape[2]. The idea leans on a property your registry already has. A map from container name to config, driven by `create` and `delete` commands, is deterministic: apply the same commands in the same order and every server ends with the identical map, with no coordination needed at apply time.

**replicated state machine** A design where identical deterministic state machines on several servers process the same commands in the same order, so they compute identical copies of the same state.

**leader** The one server in a cluster that is currently in charge of deciding the order of operations.

So the hard part shrinks to one question: how do the servers agree on the order? On each server, a **consensus** module receives commands from clients and adds them to its log, and it communicates with the modules on the other servers so that every log eventually contains the same requests in the same order, even if some servers fail[2]. Consensus on a log of commands is a clever substitution. Agreeing on every value separately means negotiating each `create` and each `delete` as its own three-way argument. Agreeing once on the contents and order of a log means the servers only ever argue about one thing, and the log then orders the commands for the replicas, which is precisely why a service keeps a log alongside its state machine state: the log is the instrument that lets replicas agree on a single execution order[1].

**consensus** The problem of getting a set of servers to agree on a single value, or a single sequence of values, despite failures and an unreliable network.

What does “agree” have to guarantee, precisely? Lamport’s statement of the requirements is the cleanest: only a value that was proposed may be chosen, only a single value is chosen, and a

process never learns that a value has been chosen unless it actually has been[3]. Notice what is missing from that list. Nothing says the answer must arrive quickly. That split, between what must always hold and what usually holds, will come back to shape everything Raft does.

> MISCONCEPTION
> **“Replication is a backup problem”**
> Copying data to more servers does protect against one disk or one machine dying. What the backup framing misses is that the copies are live and concurrent, so the hard part was never storing the data. It is deciding what happened, in what order, when the network delivers imperfect reports of the same events. A backup is a copy made after the fact. A replica is a participant in every decision.

## The world we agree in: the fault model

Before we can say whether an agreement protocol is correct, we have to fix what it is standing on, and that means being honest about how bad the network and the machines are allowed to get. This contract is called the **fault model**, and every later chapter in this topic runs inside it.

**fault model** The explicit list of failures a protocol is designed to tolerate, and the failures it assumes never happen.

The network side is worse than a first guess. Raft must maintain safety under all non-Byzantine conditions, which includes network delays, **partitions**, and packet loss, duplication, and reordering[2]. The formal specification says it bluntly: the network may reorder, drop, and duplicate messages[4]. A partition deserves special attention, because it is the failure that produces this chapter’s disaster: the network silently splits, each group looks healthy from inside, and no message between groups gets through at all.

**partition** A network failure that splits the servers into groups that can talk within the group but not across groups.

The server side has a specific shape. Servers are assumed to fail by stopping, and they may later recover from state on stable storage and rejoin the cluster[2]. This is the **crash-recovery** model, and it draws a hard line against **Byzantine** faults: Raft handles fail-stop but not Byzantine failures, so servers are assumed not to lie or collude[4]. Every message a server sends is something it genuinely believes. That assumption is doing enormous work, and this topic

**crash-recovery** A fault model where servers fail by stopping and may later restart, reloading their saved state and rejoining the system.

**Byzantine** A fault model where failed servers may behave arbitrarily, including sending false or contradictory information.

keeps faith with it throughout: nobody lies, machines just stop, lose their volatile memory, and come back.

Coming back is a designed-for event, not an afterthought. Raft retries its requests to a crashed server indefinitely, so when the server restarts, the pending request completes and it rejoins[2]. To make that safe, each server must persist enough information to stable storage to survive a restart[4], and production libraries enforce this at the API level: the etcd raft library requires that no messages be sent until the latest hard state has been persisted to disk, along with all entries from earlier batches[5]. Reboot and rejoin is a normal Tuesday.

There is one more thing Raft refuses to assume, and it surprises people: time. Raft does not depend on timing to keep the logs consistent, and faulty clocks and extreme message delays can, at worst, cause availability problems, not incorrect results[2]. Safety holds even in an asynchronous model where messages and processors proceed at arbitrary speeds[4]. Hold onto that one. By the end of this chapter you will know exactly why the designers drew that line so firmly.

## Split brain: one cluster, two truths

The natural design for three servers holding one registry is one leader plus **followers**, so let us walk that design into its failure. The leader sends periodic **heartbeats** to the followers, and a follower that stops hearing heartbeats within its timeout concludes the leader is gone and promotes itself. This failover is exactly what you would write on the first try, and it is exactly what our lab does.

**followers** The servers in a cluster that accept the leader's decisions and replicate its state.

**heartbeats** Small periodic messages a leader sends to followers to signal that it is still alive and in charge.

Here is the trap underneath it. A machine cannot distinguish a crashed server from a broken network, because the symptom is identical in both cases: no response to a query over the network[1]. The dissertation states the general form: in all practical settings it is impossible to distinguish a failed server from a slow server, and this is the key characteristic of an asynchronous system[4]. A timeout proves only that *you* stopped hearing from the peer. The peer may be dead, paused, or partitioned away from you while still talking to everyone else.

Now run the failure. The leader pauses, or a partition drops its heartbeats in one direction. A follower's timeout fires, and the follower declares itself the new leader. The lecture notes walk this exact scenario: if the backup decides the primary is down and takes over as coordinator while the primary is still running, a network partition produces **split brain**, two servers each acting as coordinator[1]. The old leader never died. It is still alive and still serving clients, and each of the two leaders now accepts writes the other never sees. Worse, since failure detectors can mistakenly report a live-but-slow leader as failed, a failover system must not just pick a new leader, it must neutralize the suspected old one: once a server has seen a larger term number, it no longer accepts requests from a leader with a smaller number[4]. Our naive design has no terms, no votes, and no way to neutralize anything.

**split brain** A failure state in which two servers in one cluster simultaneously believe they are the leader.

The lab makes this concrete with the chapter's running example. Three nodes of the container registry run as goroutines over a fake in-memory network, and the fake network can silently drop one direction of a link. The node logic that matters is tiny: a follower that has not heard a heartbeat within its election timeout simply promotes itself.

```go
    case <-tick.C:
      if n.isLeader() && time.Since(lastSent) >= heartbeat {
        for _, p := range n.Peers {
          n.Net.Send(Msg{Kind: Heartbeat, From: n.Name, To: p})
        }
        lastSent = time.Now()
      }
      if !n.isLeader() && n.sinceHeartbeat() >= electionTimeout {
        n.becomeLeader()
      }
```

A leader's write path is just as naive. It applies the write locally and fans it out to its peers, with no confirmation that anyone received anything.

```go
// Propose applies a write locally and fans it out to peers.
func (n *Node) Propose(name string, cfg Config) {
  n.Registry.Set(name, cfg)
  for _, p := range n.Peers {
    n.Net.Send(Msg{Kind: Write, From: n.Name, To: p, Name: name, Cfg: cfg})
```

```
  }
}
```

The orchestration then tells the whole story in three phases: a healthy write under leader A, a partition that drops all messages between A and the others in both directions, and a second, conflicting write served by B after B times out.

GO

```
  time.Sleep(80 * time.Millisecond)

  fmt.Println("== Phase 1: A leader, client writes web-1 cpu=2 ==")
  A.Propose("web-1", Config{CPU: 2})
  time.Sleep(60 * time.Millisecond)

  fmt.Println("== Phase 2: partition A from B,C; B times out and becomes leader ==")
  net.Drop("A", "B")
  net.Drop("A", "C")
  net.Drop("B", "A")
  net.Drop("C", "A")

  time.Sleep(220 * time.Millisecond)

  fmt.Println("== Phase 3: new client writes web-1 cpu=4 to leader B ==")
  B.Propose("web-1", Config{CPU: 4})
  time.Sleep(60 * time.Millisecond)

  fmt.Println("== Result: each node's view ==")
  for _, n := range []*Node{A, B, C} {
    role, reg := n.View()
    fmt.Printf("node %s: %s, registry %v\n", n.Name, role, reg)
  }
}
```

OUTPUT

```
== Phase 1: A leader, client writes web-1 cpu=2 ==
== Phase 2: partition A from B,C; B times out and becomes leader ==
[B] timeout: promoting self to leader
== Phase 3: new client writes web-1 cpu=4 to leader B ==
== Result: each node's view ==
node A: leader, registry map[web-1:{cpu:2}]
node B: leader, registry map[web-1:{cpu:4}]
node C: follower, registry map[web-1:{cpu:4}]
```

Read the last three lines carefully, because they are the whole chapter in miniature. Node A still believes it is the leader and holds `web-1` with `cpu:2`. Node B also believes it is the leader and holds `web-1` with `cpu:4`. Node C is a follower holding the second

value, because B's write reached it. Two leaders, two conflicting accepted writes for the same container. Notice what the network did not do. It corrupted nothing and invented nothing. It only failed to deliver messages across the partition, including B's write on its way to A, and that alone was enough to give one cluster two truths. If a client now asks any node what `web-1`'s CPU limit is, the answer depends on which node it happens to reach.

## Why the wall clock can't referee

There is a tempting fix, and it is worth taking seriously before dismissing it, because seeing exactly why it fails teaches you what the real fix must contain. Give the old leader a time limit. When B promotes itself, it stamps its leadership with the current time, and any request from a leader whose timestamp is older than the limit gets rejected. This is **fencing**, and the number being compared is a fencing token.

**fencing** Invalidating a stale leader's actions by attaching a number or timestamp to every decision and rejecting decisions that carry a stale one.

The timestamp version fails on both halves of its own machinery. First, comparing timestamps across machines requires trusting the clocks of those machines, and a lease-based fence assumes a bound on how far two machines' clocks can drift apart over a period[4]. That bound is an operational hope, not a property of hardware. Second, even a correct clock does not help, because a process can pause. Discovering and maintaining the clock-drift bound is operationally hard precisely because of scheduling and garbage collection pauses, virtual machine migrations, and clock rate adjustments for time synchronization, and if those assumptions are violated the system can return arbitrarily stale information[4]. Picture the old leader in the lab mid-GC-pause, holding a lease it believes is valid, waking to serve `delete("web-1")` with its timestamp while B is already serving `create("web-1")` with a newer one. The lease says yes. The world has moved on.

Notice that both failure paths are the timeout misconception again, wearing a clock. A lease tells you what your own clock believed about your own liveness. It cannot tell you what the rest of the cluster has decided in the meantime, and no single machine's clock is authority over a distributed decision. This is exactly why Raft refuses to put correctness in the hands of time: safety must

not depend on timing, and faulty clocks can cost at most availability, never correctness[2]. The production evidence agrees. The etcd raft library's core does not depend on clocks, and its optional lease-based read optimization is the one part that does rely on the clocks of all the machines in the group, which is why that optimization is an opt-in speedup rather than the foundation[5].

So what does Raft use instead of clocks to demote a stale leader? Numbers that the cluster counts, not time that a machine measures. Raft invalidates a stale leader with term numbers: a candidate or leader that learns its term is out of date immediately reverts to follower, and a server rejects any request carrying a stale term number[2]. In the etcd raft library, a message arriving with a higher term makes its receiver become a follower at that higher term, which is how a stale leader is forced to step down[5]. How those terms get chosen and voted on is chapter 3's subject. What this chapter needs is the shape of the answer: authority comes from something the servers count together, not from something any one of them reads off its own clock.

## No perfect answer exists, so pick your sacrifice carefully

At this point you might reasonably ask whether the whole thing is just badly engineered so far, and whether some cleverer protocol could make split brain impossible while still tolerating crashes and a messy network. The honest answer is no, and the reason is one of the famous results of the field. In 1985, Fischer, Lynch, and Paterson proved what is now called the **FLP impossibility result**: in a fully asynchronous model, consensus cannot be solved even if just a single server stops, and the result was published in the Journal of the Association for Computing Machinery in April 1985[6]. Put as a protocol constraint, no fault-tolerant consensus protocol can deterministically terminate in a purely asynchronous model[4].

**FLP impossibility result** The 1985 proof that in a fully asynchronous system, no deterministic consensus protocol can guarantee that it terminates if even one server may crash.

It is worth being precise about what this does and does not say. It does not say consensus is hopeless, and it does not say your cluster will stall forever. It says that if you demand a protocol that is guaranteed to finish in bounded time under every possible interleaving

of delays and crashes, including adversarial ones, then no deterministic protocol can give it to you. Something has to give. Every usable consensus algorithm is a choice about what to give up.

Raft's choice splits the promises in two, and this split is the most important idea in the chapter. Its safety never depends on timing: the system must not produce incorrect results just because some event happens more quickly or slowly than expected[4]. Two leaders serving conflicting writes is a safety violation, and Raft rules that out structurally, by the counting we will meet in the next section, not by hoping the timing works out. But availability, the ability to respond to clients in a timely manner, inevitably does depend on timing[4]. Raft accepts that it can occasionally stall, for example when elections collide, and buys back liveness in practice with a relaxed model of time. This idea goes by the name **partial synchrony**: rather than assume the fully asynchronous world where anything can take any amount of time, Raft assumes the real world, which is usually well behaved for long stretches.

**partial synchrony** A system model that assumes timing is not fully arbitrary, only that message delays and processing times eventually stay within some loose bound.

Concretely, Raft requires broadcast time to be much less than the election timeout, which in turn must be much less than the **mean time between failures**[2]. Each link in the chain does a specific job. Keeping broadcast time an order of magnitude below the election timeout means heartbeats reliably arrive before followers time out, and it also makes split votes unlikely. Keeping the election timeout a few orders of magnitude below the mean time between failures means the system makes steady progress rather than spending its life re-electing leaders[4]. When the world cooperates, and in practice it usually does, Raft finishes. When it does not, Raft is merely slow, never wrong. And because FLP rules out a deterministic guarantee, chapter 3's elections use randomized timeouts to break ties, which is exactly the kind of sacrifice the result forces.

**mean time between failures** The average time a machine or system runs before its next failure.

## Majorities: the counting trick

We are left with the real question. If the network can partition, if timeouts cannot be trusted, and if clocks are out, what mechanism can possibly let a cluster make a decision that survives the failure

of the decider? The answer is arithmetic, and you can verify it yourself.

Raft requires that a decision be confirmed by a **majority** of the servers, a **quorum**: an entry counts as committed only once the leader that created it has replicated it on a majority of the servers[2]. The magic is in what any two majorities share. Count it for three servers. A majority is at least two, and any two groups of two out of three must overlap, since two plus two is four but there are only three servers. In general, in a cluster of $n$ servers a majority is $\lfloor n/2 \rfloor + 1$[7], and any two majorities therefore share at least one server. This is **quorum intersection**, and the shared server is the witness that carries information forward.

**majority** More than half of the servers in a cluster.

**quorum** The minimum number of servers that must agree for a cluster decision to count.

**quorum intersection** The property that any two majorities of a cluster must share at least one common server.

Here is how that witness earns its pay, as a preview of chapter 4. Suppose an old leader replicated an entry on a majority, and then died. A new leader cannot be elected without collecting votes from a majority. The old leader's majority and the new leader's majority overlap, so at least one server both accepted the entry from the old leader and voted for the new one[2]. Whatever that server knows, the new leader can be made to learn, which is how a committed entry survives later elections instead of being quietly forgotten. Split brain dies here: two conflicting decisions would each need a majority, and two majorities cannot be disjoint, so the two truths from the lab cannot both be confirmed.

The arithmetic also tells you how many machines to buy, and this is the operator-facing version of the same counting. For a cluster with $n$ members the quorum is $\lfloor n/2 \rfloor + 1$, which gives the failure-tolerance table etcd publishes: a 3-member cluster tolerates 1 failure, a 5-member cluster tolerates 2, and a 7-member cluster tolerates 3[7]. The table has a quiet lesson in it. Compare a 4-member cluster, which needs a quorum of 3 and tolerates 1 failure, with a 3-member cluster, which needs a quorum of 2 and also tolerates 1. The fourth machine bought you nothing but a bigger quorum. That is why etcd recommends an odd number of members: an odd-size cluster tolerates the same number of failures as the next even size but with fewer nodes[7]. There is no hard limit on cluster size, but an etcd cluster probably should have no more than seven nodes[7], so three, five, or seven is the practical menu.

TRY IT

Run the lab yourself, from `topics/raft/chapters/01-why-consensus/lab/split-brain/`, with `go run split-brain.go`. Then change one thing: in `main`, also drop B's heartbeats to C before the phase 3 write, so B is fully partitioned from everyone. Predict what the result lines will show before you run it. The registry state machine the lab runs is the one below, the running example of the whole chapter in thirteen lines.

GO

```go
// Registry is the state machine: a map from container name to config.
type Registry struct {
  mu sync.Mutex
  m  map[string]Config
}

func NewRegistry() *Registry { return &Registry{m: make(map[string]Config)} }

func (r *Registry) Set(name string, c Config) {
  r.mu.Lock()
  defer r.mu.Unlock()
  r.m[name] = c
}
```

Chapters 3 and 4 turn this counting trick into the two halves of Raft: elections that grant leadership only to a candidate with a majority of votes, and commits that confirm a log entry only on a majority of servers. Everything else, terms, heartbeats, log matching, is bookkeeping in service of that one piece of arithmetic.

## Check yourself

1. In the lab, node C ends up a follower holding B's value. Using only what the fake network dropped and the two election timeouts, explain why C never promoted itself, and why it holds `cpu:4` rather than `cpu:2`.
2. Suppose you "fix" the lab by having the new leader B broadcast a message announcing its leadership, and having any node that receives it stop following the old leader. Describe a partition pattern in which this still ends in split brain.

3. A 9-member cluster loses 4 servers in quick succession, then a client asks for a write. What exactly happens to the write, and is that a safety failure or an availability failure? Which of the two can Raft's design never allow?
4. FLP says no deterministic protocol can guarantee termination in a fully asynchronous system with one crash. Raft terminates anyway, most of the time. What, precisely, did Raft give up, and what did it assume to buy that back?
5. Your teammate proposes timestamps as fencing tokens because "clock skew between two machines in one rack is tiny." Give the two distinct reasons this argument fails, one about clocks and one about pauses.

Answers are at the end of the chapter.

## Answers

1. The partition drops every message between A and the other two, so after it neither B nor C hears from A. B's election timeout is 150 ms and C's is 250 ms, so B's timer runs out first and B promotes itself. B's heartbeats still reach C and keep resetting C's timer, so C never promotes itself. C did apply A's first write, `cpu=2`, before the partition. B's later write, `cpu=4`, reached C and replaced it, while the same write on its way to A was dropped. Neither B nor C can tell whether A crashed or is merely cut off, which is the failed-versus-slow problem in miniature.
2. Split the cluster into two groups, A plus C and B alone, dropping every message between the groups. B times out and broadcasts its announcement, but the broadcast can only reach servers in B's group, which is nobody. A and C never hear it, A keeps leading C, and each side confirms its own leader. Broadcasting does not help when the partition blocks the broadcast itself.
3. A quorum of 9 is 5, and only 5 servers remain, so a majority still exists and the write can commit, on exactly 5 servers. Had one more server failed, no majority would exist, the write could not be confirmed, and the cluster would refuse until servers recovered. That refusal is an availability failure. The

safety property, never returning an incorrect result, is the one Raft's design never sacrifices, because safety rests on quorum intersection rather than on timing.

4. Raft gave up guaranteed termination, which is a liveness property, and kept safety unconditionally, so unlucky timing can make it stall but never make it wrong. It assumed partial synchrony, expressed as the timing requirement that broadcast time be much less than the election timeout, which in turn is much less than the mean time between failures, so elections settle and progress resumes in practice.
5. First, comparing timestamps across machines assumes a bound on clock drift between them, and no such bound is a property of the hardware, it is an operational assumption that can be violated by clock rate adjustments and migrations. Second, a process can pause during garbage collection or scheduling and wake believing its lease is still valid, serving stale decisions with a timestamp that was correct when the process fell asleep. In both cases the old leader's own view is internally consistent and wrong about the world.

## Sources

[1] 6.5840 Lecture 6: RSM and Raft (1), 6.5840 (6.824) spring 2026 course notes, Lecture 6
https://pdos.csail.mit.edu/6.824/notes/l-raft.txt

[2] In Search of an Understandable Consensus Algorithm (Extended Version), Extended version published 2014-05-20 (USENIX ATC '14 paper plus extra material)
https://raft.github.io/raft.pdf

[3] Paxos Made Simple, 2001-11-01 (SIGACT News 32(4), December 2001)
https://lamport.azurewebsites.net/pubs/paxos-simple.pdf

[4] Consensus: Bridging Theory and Practice (PhD dissertation), Stanford University, August 2014; dissertation source commit 0c10ea1b (2016-05-24)
https://raw.githubusercontent.com/ongardie/dissertation/0c10ea1b53d73bf6d163bdde8e4141a1cbd8f36a/online.pdf

[5] etcd-io/raft (Go Raft library), tag v3.7.0 (commit b867cf13f6bc0dae21204302df97bc2355c3af55)
https://github.com/etcd-io/raft/tree/v3.7.0

[6] CS 2620 lecture notes: Fischer-Lynch-Paterson impossibility, Harvard CS 2620 lecture notes, page read 2026-10-11
https://read.seas.harvard.edu/cs2620/2026/lectures/flp/

[7] etcd documentation: FAQ (quorum, membership, failure tolerance), etcd v3.5 docs, page last modified 2024-12-05
https://etcd.io/docs/v3.5/faq/